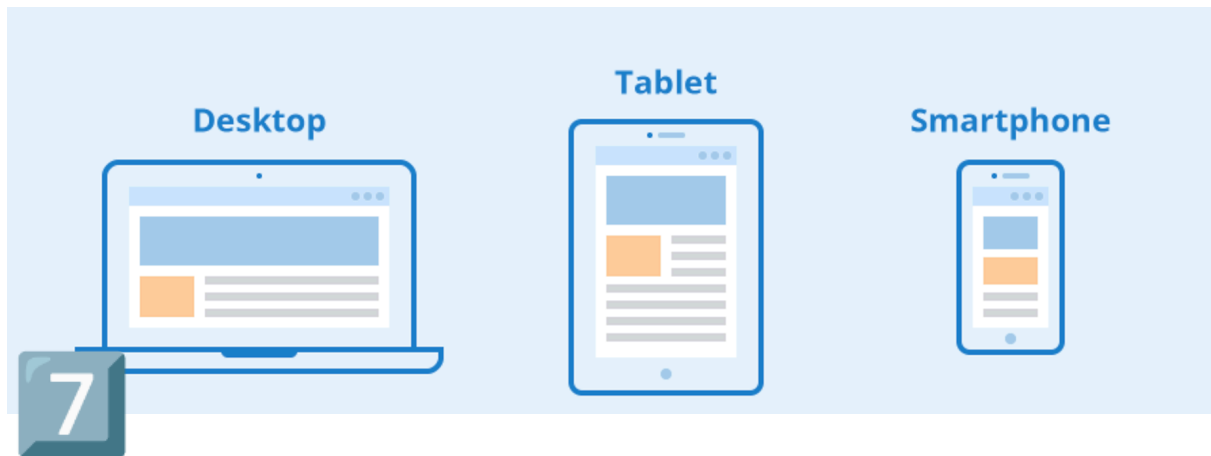




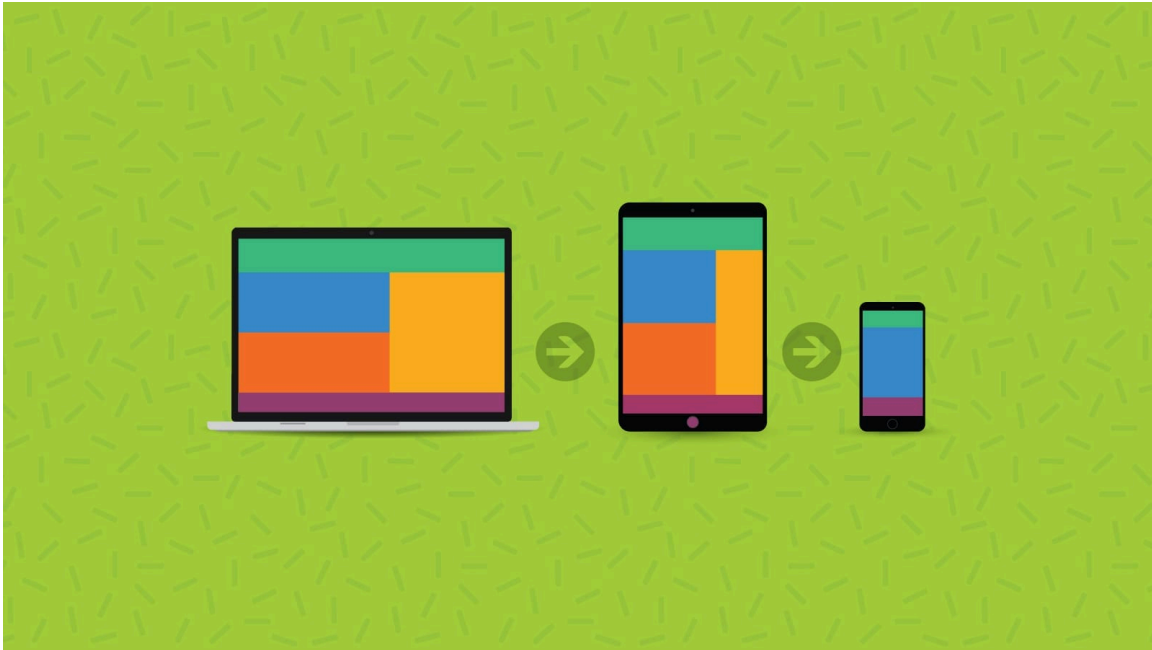
Day 7: Media Queries & Positions

Introduction:

- Media queries are what make modern responsive design possible. With them, you can set different styling based on things like a users screen size, device capabilities, or user preferences.

What are media queries?

- What is a media query? A media query is a specific feature of CSS that lets you conditionally apply styling based on a media type, a media feature or both.
- You use them primarily to check the screen dimensions and apply CSS based on that, but media queries can do many other powerful things.



- A media query is an expression that evaluates to either True or False.
- A media query consists of a test, followed by as many CSS rules as we want, with the rule block enclosed in a set of braces. If the test condition is false, the browser simply ignores the rule block and moves on.

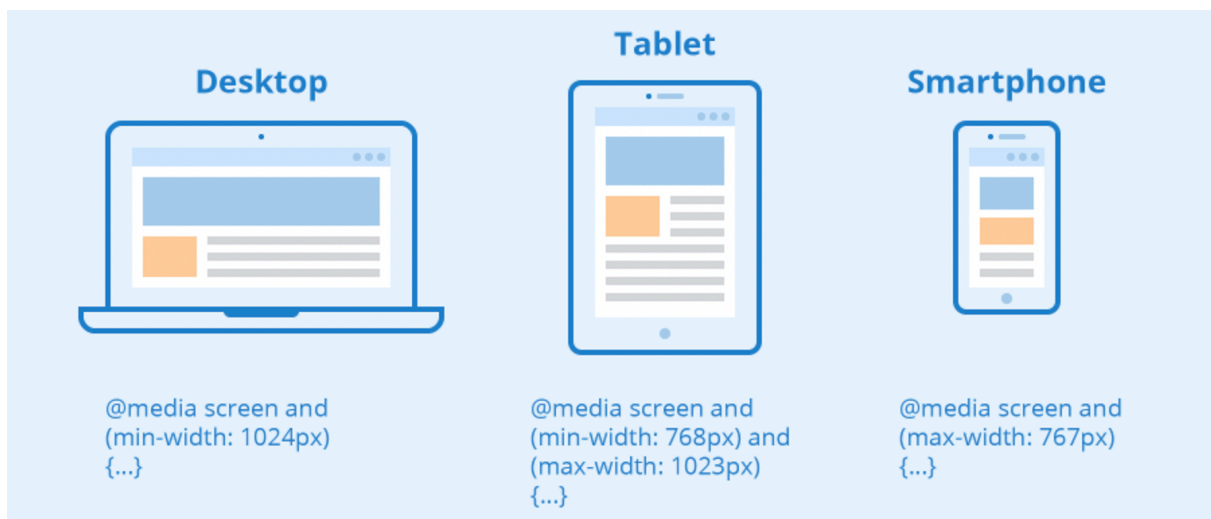
Syntax of Media Query

@media screen (min-width: 320px) and (max-width: 768px)

AT-RULE MEDIA TYPE MEDIA FEATURE OPERATOR MEDIA FEATURE

- **@media** specifies media query declaration.
- **Media type** specifies what type of media are we trying to target, In many cases, you'll see a **screen** value used here, which makes sense since many of the media types we're trying to match are devices with screens attached to them. It also includes
 - **all**: Matches all devices

- `print` : Matches documents that are viewed in a print preview or any media that breaks the content up into pages intended to print.
- `screen` : Matches devices with a screen
- **Media Feature** defines what features we are trying to match it to.
 - Desktop: `(min-width:1024px)`
 - Tablet: `(min-width:768px)` and `(max-width:1023px)`
 - Smartphone : `(min-width:340px)` and `(max-width:767px)`



```

/* large screens */
@media all and (min-width: 1024px) {
  /* write code here */
}

/* medium screens */
@media all and (min-width: 768px) and (max-width: 1024px) {
  /* write code here */
}

/* small screens */
@media all and (min-width: 320px) and (max-width: 767px) {
  /* write code here */
}

```

- Let's see how media query works by taking a small example.
- In this example, we are just giving background-color property to the body, by default whatever we write is for large screens.

```
<html>
  <head>
    <title>Document</title>
    <style>
      body {
        background-color: teal;
      }
    </style>
  </head>
  <body></body>
</html>
```

- **Medium Screens**

```
<html>
  <head>
    <title>Document</title>
    <style>
      body {
        background-color: teal;
      }
      @media all and (min-width: 768px) and (max-width: 1024px) {
        body {
          background-color: pink;
        }
      }
    </style>
  </head>
  <body></body>
</html>
```

- **Small Screens:**

```

<html>
  <head>
    <title>Document</title>
    <style>
      body {
        background-color: teal;
      }

      /* meidum screens */
      @media all and (min-width: 768px) and (max-widt
h: 1024px) {
        body {
          background-color: pink;
        }
      }

      /* small screens */
      @media all and (min-width: 320px) and (max-width: 767
px) {
        body {
          background-color: brown;
        }
      }

    </style>
  </head>
  <body></body>
</html>

```

Output Video for all screens

<https://s3-us-west-2.amazonaws.com/secure.notion-static.com/800546f4-3503-406f-8395-94f2fe8e2db6/mq.mp4>

- In the above example we are just changing background color, we can also change whatever property we want for particular screen size.

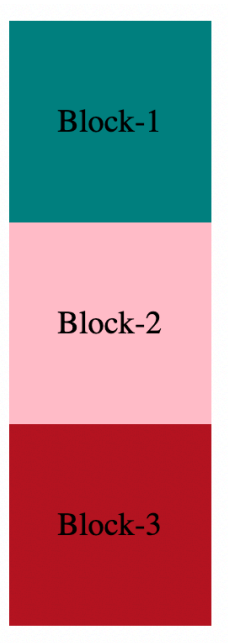
CSS Positions:

Overview:

- The CSS position property defines the position of an HTML element inside the browser's viewport. By default, all the HTML elements will follow a default order (stack)
- We can use the CSS position property to manipulate the position of HTML elements inside the viewport (top, right, bottom, and left CSS position)
- By default, the browser renders the HTML elements one after the other in the viewport. The ordering can be horizontal or vertical, depending on the element type.
 - Block elements: Vertical alignment

```
<body>
  <div>Block-1</div>
  <div>Block-2</div>
  <div>Block-3</div>
</body>
```

Output:



- Inline elements: Horizontal alignment

```
<body>
  <span>Block-1</span>
  <span>Block-2</span>
  <span>Block-3</span>
</body>
```



CSS Position Types

1. Static Positioning
2. Relative Positioning
3. Fixed Positioning
4. Absolute Positioning
5. Sticky Positioning

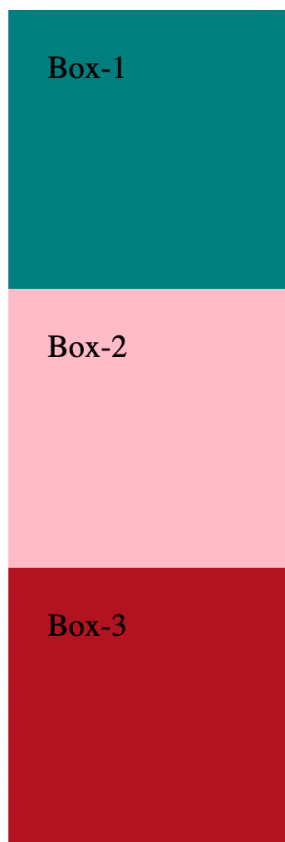
Static Position

- All HTML elements are positioned static by default.
- With static positioning, the elements are positioned along with the natural flow of the document.
- The properties left, right, top, bottom, and z-index do not affect the elements positioned as static.

```
<html>
  <head>
    <style>
      #teal {
        background-color: teal;
        position: static;
        left: 50px;
      }
      #pink {
        background-color: pink;
      }
      #brown {
        background-color: brown;
      }
    </style>
  </head>
  <body>
    <div id="teal">Box-1</div>
    <div id="pink">Box-2</div>
    <div id="brown">Box-3</div>
  </body>
</html>
```

- We set the position property to static and the left property to 50px for the teal div.

Output:



- Eventhough the left property is set to 50px for the teal div, it doesn't alter its position in the viewport.

Live Code : [Codepen](#)

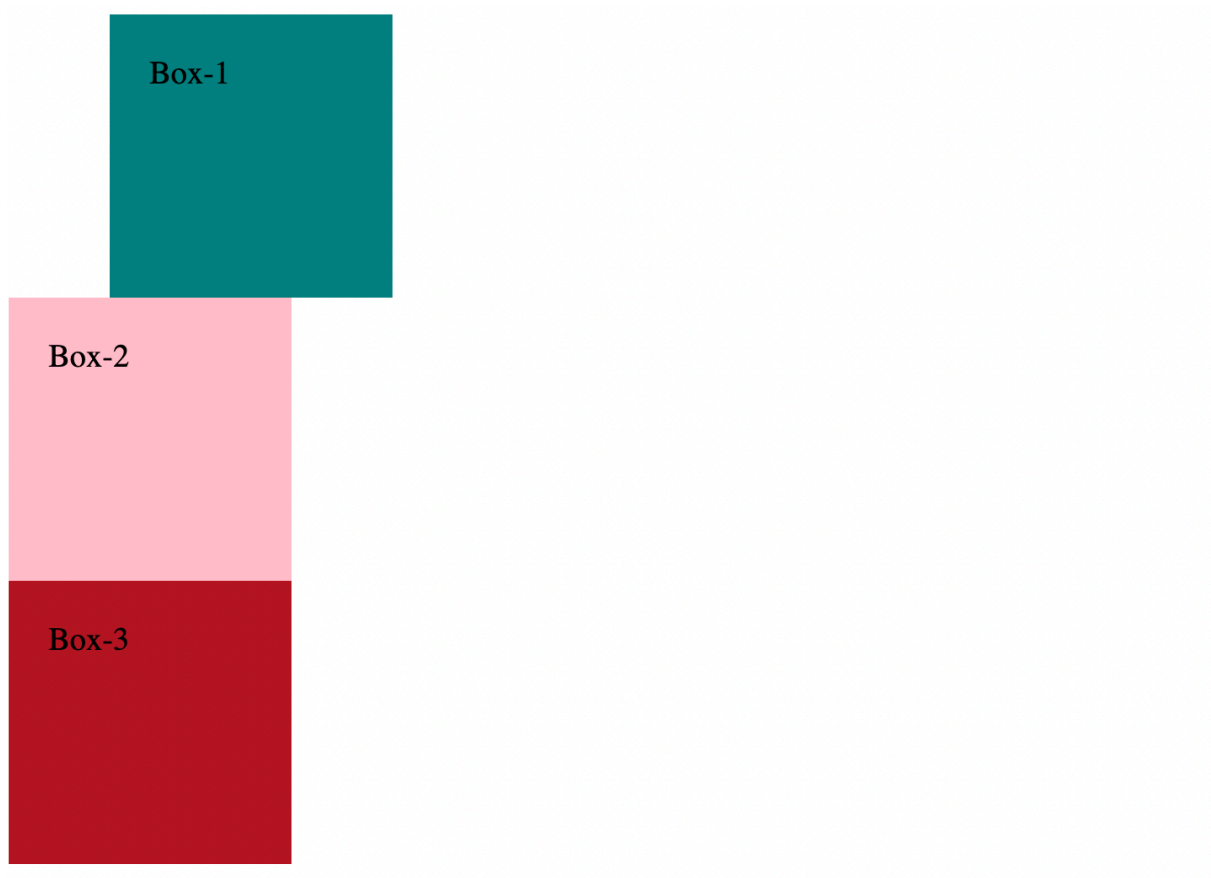
Relative Position

- With relative positioning, the elements are positioned along with the natural flow of the document. But unlike static elements, the position of the relative elements can be modified using left, right, top, bottom, and z-index properties.
- Now for the same HTML content we saw above, we are setting the position property to relative and the left property to 50px for the teal div

```
<html>
  <head>
    <style>
      #teal {
        background-color: teal;
        position: relative;
```

```
        left: 50px;
    }
    #pink {
        background-color: pink;
    }
    #brown {
        background-color: brown;
    }
</style>
</head>
<body>
    <div id="teal">Box-1</div>
    <div id="pink">Box-2</div>
    <div id="brown">Box-3</div>
</body>
</html>
```

Output:



- We can see that the teal div is moved 50px from the left.
- Even though it is moved from the left, there is a space between the red and the green div. This is because the relative positioning **doesn't remove the element from its default position**
- Instead, the element is moved to the given position and its default place left unoccupied.

Live Code : [Codepen](#)

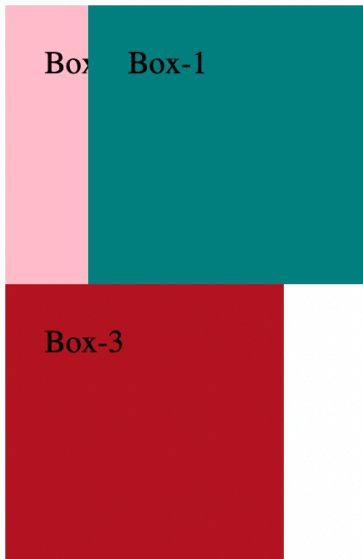
Absolute Position:

- The elements are positioned relative to their parent elements with absolute positioning. The absolute elements are positioned relative to the closest ancestor with any position property other than static.
- If the closest ancestor has a static position, the element is positioned relative to the next parent element without the static position property.

```
<html>
  <head>
    <style>
      #teal {
        background-color: teal;
        position: absolute;
        left: 50px;
      }
      #pink {
        background-color: pink;
      }
      #brown {
        background-color: brown;
      }
    </style>
  </head>
  <body>
    <div id="teal">Box-1</div>
    <div id="pink">Box-2</div>
    <div id="brown">Box-3</div>
```

```
</body>
</html>
```

Output:



- We can see that the teal div is moved 200px from the left.
- But unlike relative elements, the absolute element is removed from its default position and moved to the new position. Hence its old position is occupied by the next element.

Live Code : [Codepen](#)

Fixed Position

- Elements with fixed positioning are always positioned relative to the HTML element (i.e.) the root of the document. The fixed elements stay in the same position irrespective of the scrolling.

```
<style>
  div.fixed {
    position: fixed;
    bottom: 20px;
    right: 0;
  }
  body {
```

```
        height: 900px;
    }
</style>
</head>
<body>
    <div class="fixed">?</div>
</body>
```

Output



Live Code: [Codepen](#)

Sticky Position

- With sticky positioning, the element behaves like a relative positioned until a certain scroll point, and then it will be fixed.

```
<style>
  .sticky {
    position: -webkit-sticky;
    position: sticky;
    top: 20px;
    padding: 5px;
    background-color: #cae8ca;
  }
</style>
</head>
<body>
  <div class="sticky">I am sticky!</div>

  <div style="padding-bottom: 2000px">
    Lorem ipsum dolor sit amet, consectetur adipisicing e
    lit. Minus, assumenda
      voluptas dicta amet ipsam, neque molestias eaque susc
    ipit mollitia
      architecto voluptatibus maxime veniam ducimus consect
    etur non porro eos
      vel omnis? Culpa, doloribus quas! Error dolor labore
    sit, unde earum velit
      deleniti maiores ex totam dignissimos obcaecati minim
    a numquam eius a
      harum fugit veritatis possimus, animi similique natu
    s! Accusamus, neque
      voluptates! Voluptas corrupti exercitationem error qu
    i tenetur asperiores
      eveniet facilis atque laborum porro soluta similique,
    beatae reiciendis
      rem possimus voluptatibus fuga id voluptate aperiam,
    iusto maxime cum
      dignissimos distinctio eaque!
  </div>
</body>
```

<https://s3-us-west-2.amazonaws.com/secure.notion-static.com/8c2059fc-e1d1-4e7b-8cb0-8c5c0b39c676/sticky.mp4>

Live Code: Codepen